Операционные системы

Анализ файловой структуры UNIX. Команды для работы с файлами и каталогами

Албаша Али Омран

2026-03-13

Содержание (i)

Цели и задачи работы

Процесс выполнения лабораторной работы

Выводы по проделанной работе

1. Цели и задачи работы

Цель лабораторной работы

Ознакомление с файловой системой Linux, её структурой, именами и содержанием каталогов. Приобретение практических навыков по применению команд для работы с файлами и каталогами, по управлению процессами, по проверке использования диска и обслуживанию файловой системы.

Задачи лабораторной работы

1 Выполнить приимеры

2 Выполнить дествия по работе с каталогами и файлами

3 Выполнить действия с правами доступа

4 Получить дополнительные сведения при помощи справки по командам.

2. Процесс выполнения лабораторной работы

Выполнение примеров

```
alialbasha@alialbasha:~$ touch abc1
alialbasha@alialbasha:~$ cp abc1 april
alialbasha@alialbasha:~$ cp abc1 may
alialbasha@alialbasha:~$ mkdir monthly
alialbasha@alialbasha:~$ cp april may monthly
alialbasha@alialbasha:~$ cp monthly/may monthly/june
alialbasha@alialbasha:~$ ls monthly
april  june  may
alialbasha@alialbasha:~$ mkdir monthly.00
alialbasha@alialbasha:~$ cp -r monthly monthly.00
alialbasha@alialbasha:~$ cp -r monthly.00 /tmp
alialbasha@alialbasha:~$
```

Рисунок 1: Выполнение примеров

Выполнение примеров

```
alialbasha@alialbasha:~$ mv april july
alialbasha@alialbasha:~$
^[[200~mv july monthly.00
^[[201~alialbasha@alialbamv july monthly.00thly.00
alialbasha@alialbasha:~$ ls monthly.00
july  monthly
alialbasha@alialbasha:~$ mv monthly.00 monthly.01
alialbasha@alialbasha:~$ mkdir reports
alialbasha@alialbasha:~$ mv monthly.01 reports
alialbasha@alialbasha:~$ mv reports/monthly.01 reports/monthly
alialbasha@alialbasha:~$
```

Рисунок 2: Выполнение примеров

Выполнение примеров

```
alialbasha@alialbasha:~$
alialbasha@alialbasha:~$ touch may
alialbasha@alialbasha:~$ ls -l may
-rw-r--r--. 1 alialbasha alialbasha 0 Mar 13 15:51 may
alialbasha@alialbasha:~$ chmod u+x may
alialbasha@alialbasha:~$ ls -l may
-rwxr--r--. 1 alialbasha alialbasha 0 Mar 13 15:51 may
alialbasha@alialbasha:~$ chmod u-x may
alialbasha@alialbasha:~$ ls -l may
-rw-r--r--. 1 alialbasha alialbasha 0 Mar 13 15:51 may
alialbasha@alialbasha:~$ chmod g-r,o-r monthly
alialbasha@alialbasha:~$ chmod g+w abc1
alialbasha@alialbasha:~$
```

Рисунок 3: Выполнение примеров

Создание директорий и копирование файлов

```
alialbasha@alialbasha:~$
alialbasha@alialbasha:~$ cp /usr/include/linux/sysinfo.h ~
alialbasha@alialbasha:~$ mv sysinfo.h equipment
alialbasha@alialbasha:~$ mkdir ski.plases
alialbasha@alialbasha:~$ mv equipment ski.plases/
alialbasha@alialbasha:~$
alialbasha@alialbasha:~$ mv ski.plases/equipment ski.plases/equiplist
alialbasha@alialbasha:~$ touch abc1
alialbasha@alialbasha:~$ cp abc1 ski.plases/equiplist2
alialbasha@alialbasha:~$ cd ski.plases/
alialbasha@alialbasha:~/ski.plases$ mkdir equipment
alialbasha@alialbasha:~/ski.plases$ mv equiplist equipment/
alialbasha@alialbasha:~/ski.plases$ mv equiplist2 equipment/
alialbasha@alialbasha:~/ski.plases$ cd
alialbasha@alialbasha:~$ mkdir newdir
alialbasha@alialbasha:~$ mv newdir ski.plases/
alialbasha@alialbasha:~$ mv ski.plases/newdir/ ski.plases/plans
alialbasha@alialbasha:~$
```

Рисунок 4: Работа с каталогами

Работа с командой chmod

```
alialbasha@alialbasha:~$ mkdir australia play
alialbasha@alialbasha:~$ touch my_os feathers
alialbasha@alialbasha:~$ chmod 744 australia/
alialbasha@alialbasha:~$ chmod 711 play/
alialbasha@alialbasha:~$ chmod 544 my_os
alialbasha@alialbasha:~$ chmod 664 feathers
alialbasha@alialbasha:~$ ls -l
total 0
-rw-rw-r--. 1 alialbasha alialbasha  0 Mar 13 15:52 abc1
drwxr--r--. 1 alialbasha alialbasha  0 Mar 13 15:52 australia
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Desktop
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Documents
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Downloads
-rw-rw-r--. 1 alialbasha alialbasha  0 Mar 13 15:52 feathers
drwxr-xr-x. 1 alialbasha alialbasha 74 Feb 23 18:39 git-extended
-rw-r--r--. 1 alialbasha alialbasha  0 Mar 13 15:51 may
drwx--x--x. 1 alialbasha alialbasha 24 Mar 13 15:50 monthly
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Music
-r-xr--r--. 1 alialbasha alialbasha  0 Mar 13 15:52 my_os
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Pictures
drwx--x--x. 1 alialbasha alialbasha  0 Mar 13 15:52 play
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Public
drwxr-xr-x. 1 alialbasha alialbasha 14 Mar 13 15:51 reports
drwxr-xr-x. 1 alialbasha alialbasha 28 Mar 13 15:52 ski.plases
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Templates
drwxr-xr-x. 1 alialbasha alialbasha  0 Feb 23 18:18 Videos
drwxr-xr-x. 1 alialbasha alialbasha 10 Feb 23 18:28 work
```

Файл /etc/passwd

```
ftp:x:14:50:FTP User:/var/ftp:/usr/sbin/nologin
nobody:x:65534:65534:Kernel Overflow User:/:/usr/sbin/nologin
dbus:x:81:81:System Message Bus:/:/usr/sbin/nologin
tss:x:59:59:Account used for TPM access:/:/usr/sbin/nologin
systemd-oom:x:999:999:systemd Userspace OOM Killer:/:/usr/sbin/nologin
polkitd:x:114:114:User for polkitd:/:/sbin/nologin
systemd-coredump:x:998:998:systemd Core Dumper:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
chrony:x:996:996:chrony system user:/var/lib/chrony:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/:/usr/sbin/nologin
systemd-resolve:x:193:193:systemd Resolver:/:/usr/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/usr/sbin/nologin
unbound:x:993:993:Unbound DNS resolver:/var/lib/unbound:/sbin/nologin
clevis:x:992:992:Clevis Decryption Framework unprivileged user:/var/cache/clevis:/usr/sbin/nologin
usbmuxd:x:113:113:usbmuxd user:/:/usr/sbin/nologin
gluster:x:991:991:GlusterFS daemons:/run/gluster:/usr/sbin/nologin
qemu:x:107:107:qemu user:/:/usr/sbin/nologin
apache:x:48:48:Apache:/usr/share/httpd:/sbin/nologin
openvpn:x:990:990:OpenVPN:/etc/openvpn:/usr/sbin/nologin
nm-openvpn:x:989:989:Default user for running openvpn spawned by NetworkManager:/:/usr/sbin/nologin
abrt:x:173:173::/etc/abrt:/usr/sbin/nologin
wsdd:x:987:987:Web Services Dynamic Discovery host daemon:/:/usr/sbin/nologin
nm-openconnect:x:986:986:NetworkManager user for OpenConnect:/:/usr/sbin/nologin
/etc/passwd
```

Рисунок 6: Файл /etc/passwd

```
alialbasha@alialbasha:~$ cp feathers file.old
alialbasha@alialbasha:~$ mv file.old play/
alialbasha@alialbasha:~$ mkdir fun
alialbasha@alialbasha:~$ cp -R play/ fun/
alialbasha@alialbasha:~$ mv fun/ play/games
alialbasha@alialbasha:~$ chmod u-r feathers
alialbasha@alialbasha:~$ cat feathers
cat: feathers: Permission denied
alialbasha@alialbasha:~$ cp feathers feathers2
cp: cannot open 'feathers' for reading: Permission denied
alialbasha@alialbasha:~$ chmod u+r feathers
alialbasha@alialbasha:~$ chmod u-x play/
alialbasha@alialbasha:~$ cd play/
bash: cd: play/: Permission denied
alialbasha@alialbasha:~$ chmod +x play/
alialbasha@alialbasha:~$
```

Рисунок 7: Работа с файлами и правами доступа

Справка по командам

```
MOUNT(8)                                System Administration                                MOUNT(8)

NAME
       mount - mount a filesystem

SYNOPSIS
       mount [-h|-V]

       mount [-l] [-t fstype]

       mount -a [-fFnrsvw] [-t fstype] [-O optlist]

       mount [-fnrsvw] [-o options] device|mountpoint

       mount [-fnrsvw] [-t fstype] [-o options] device mountpoint

       mount --bind|--rbind|--move olddir newdir

       mount --make-[shared|slave|private|unbindable|rshared|rslave|rprivate|runbindable] mountpoint

DESCRIPTION
       All files accessible in a Unix system are arranged in one big tree, the file hierarchy, rooted at /.
       These files can be spread out over several devices. The mount command serves to attach the filesystem
       found on some device to the big file tree. Conversely, the umount(8) command will detach it again. The
       filesystem is used to control how data is stored on the device or provided in a virtual way by network
       or other services.

       The standard form of the mount command is:

          mount -t type device dir

       This tells the kernel to attach the filesystem found on device (which is of type type) at the
       directory dir. The option -t type is optional. The mount command is usually able to detect a
       filesystem. The root permissions are necessary to mount a filesystem by default. See section
 Manual page mount(8) line 1 (press h for help or q to quit)
```

Рисунок 8: Команда mount

Справка по командам

```
FSCK(8)                                System Administration                                FSCK(8)

NAME
       fsck - check and repair a Linux filesystem

SYNOPSIS
       fsck [-lsAVRTMNP] [-r [fd]] [-C [fd]] [-t fstype] [filesystem...] [--] [fs-specific-options]

DESCRIPTION
       fsck is used to check and optionally repair one or more Linux filesystems. filesystem can be a device
       name (e.g., /dev/hdc1, /dev/sdb2), a mount point (e.g., /, /usr, /home), or a filesystem label or UUID
       specifier (e.g., UUID=8868abf6-88c5-4a83-98b8-bfc24057f7bd or LABEL=root). Normally, the fsck program
       will try to handle filesystems on different physical disk drives in parallel to reduce the total
       amount of time needed to check all of them.

       If no filesystems are specified on the command line, and the -A option is not specified, fsck will
       default to checking filesystems in /etc/fstab serially. This is equivalent to the -As options.

       The exit status returned by fsck is the sum of the following conditions:

       0
           No errors

       1
           Filesystem errors corrected

       2
           System should be rebooted

       4
           Filesystem errors left uncorrected

       8
           Operational error
 Manual page fsck(8) line 1 (press h for help or q to quit)
```

Рисунок 9: Команда fsck

Справка по командам

```
MKFS(8)                                System Administration                                MKFS(8)

NAME
       mkfs - build a Linux filesystem

SYNOPSIS
       mkfs [options] [-t type] [fs-options] device [size]

DESCRIPTION
       This mkfs frontend is deprecated in favour of filesystem specific mkfs.<type> utils.

       mkfs is used to build a Linux filesystem on a device, usually a hard disk partition. The device
       argument is either the device name (e.g., /dev/hda1, /dev/sdb2), or a regular file that shall contain
       the filesystem. The size argument is the number of blocks to be used for the filesystem.

       The exit status returned by mkfs is 0 on success and 1 on failure.

       In actuality, mkfs is simply a front-end for the various filesystem builders (mkfs.fstype) available
       under Linux. The filesystem-specific builder is searched for via your PATH environment setting only.
       Please see the filesystem-specific builder manual pages for further details.

OPTIONS
       -t, --type type
           Specify the type of filesystem to be built. If not specified, the default filesystem type
           (currently ext2) is used.

       fs-options
           Filesystem-specific options to be passed to the real filesystem builder.

       -V, --verbose
           Produce verbose output, including all filesystem-specific commands that are executed. Specifying
           this option more than once inhibits execution of any filesystem-specific commands. This is really
           only useful for testing.

 Manual page mkfs(8) line 1 (press h for help or q to quit)
```

Рисунок 10: Команда mkfs

Справка по командам

```
KILL(1)                                User Commands                                KILL(1)

NAME
       kill - terminate a process

SYNOPSIS
       kill  [-signal|-s signal|-p]  [-q value] [-a] [--timeout milliseconds signal] [--] pid|name...

       kill -l [number|0xsigmask] | -L

       kill -d pid

DESCRIPTION
       The command kill sends the specified signal to the specified processes or process groups.

       If no signal is specified, the TERM signal is sent. The default action for this signal is to terminate
       the process. This signal should be used in preference to the KILL signal (number 9), since a process
       may install a handler for the TERM signal in order to perform clean-up steps before terminating in an
       orderly fashion. If a process does not terminate after a TERM signal has been sent, then the KILL
       signal may be used; be aware that the latter signal cannot be caught, and so does not give the target
       process the opportunity to perform any clean-up before terminating.

       Most modern shells have a builtin kill command, with a usage rather similar to that of the command
       described here. The --all, --pid, and --queue options, and the possibility to specify processes by
       command name, are local extensions.

       If signal is 0, then no actual signal is sent, but error checking is still performed.

ARGUMENTS
       The list of processes to be signaled can be a mixture of names and PIDs.

       pid
           Each pid can be expressed in one of the following ways:

 Manual page kill(1) line 1 (press h for help or q to quit)
```

Рисунок 11: Команда kill

3. Выводы по проделанной работе

Вывод

В ходе данной работы мы ознакомились с файловой системой Linux, её структурой, именами и содержанием каталогов. Научились совершать базовые операции с файлами, управлять правами их доступа для пользователя и групп. Ознакомились с Анализом файловой системы. А также получили базовые навыки по проверке использования диска и обслуживанию файловой системы.